

Devi AI DevOps Platform

An AI-Powered Infrastructure Automation Operating System

A Project Report submitted for the award of

Bachelor of Technology in
Computer Science and Engineering

Submitted by

Project Team

Under the guidance of

Prof. Guide Name

Department of Computer Science and Engineering

University / Institute Name

May 21, 2026

Certificate

This is to certify that the project titled “**Devi AI DevOps Platform — An AI-Powered Infrastructure Automation Operating System**” is a bonafide work carried out by the Project Team in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** during the academic year 2025–2026.

Prof. Guide Name

(Project Guide)

Head of Department

(CSE)

External Examiner

Date: May 21, 2026 Place: City

Abstract

Devi is a production-grade, AI-powered DevOps automation platform that transforms infrastructure management through natural language interaction. Built with a futuristic interface inspired by Iron Man's Jarvis, it enables users to deploy, monitor, and manage Docker containers and Kubernetes clusters using voice commands or text input. The platform integrates an NLP engine for command parsing, a conversational voice assistant with silence-based auto-submission, real-time system monitoring with anomaly detection, and an intelligent safety layer enforcing image whitelists and operation timeouts. All container operations execute against the real Docker daemon when available, with graceful mock fallbacks.

Keywords: DevOps, AI, Docker, Kubernetes, NLP, Container Orchestration, Voice Commands, Infrastructure Automation

Acknowledgement

We extend our sincere gratitude to our project guide **Prof. Guide Name** for their invaluable guidance, and to the **Head of Department** for providing necessary resources. We thank the open-source community for the tools that made this platform possible — Next.js, React, Express.js, Dockerode, and Socket.io. Finally, we thank our families for their unwavering support.

— **Project Team**

Contents

1	Introduction	1
1.1	Overview	1
1.2	Motivation	1
1.3	Objectives	1
1.4	Scope	1
2	System Architecture	2
2.1	Three-Tier Architecture	2
2.2	Service Ports	2
2.3	Technology Stack	2
3	Key Features	3
3.1	Natural Language Management	3
3.2	AI Command Parser	3
3.3	Safe Execution Layer	3
3.4	Anomaly Detection	3
3.5	Voice Assistant	4
3.6	Real-Time Deployment Streaming	4
3.7	Deployment Workflow	4
3.8	Frontend Components	4
4	API Reference	6
4.1	Docker Endpoints	6
4.2	Kubernetes Endpoints	6
4.3	AI and System Endpoints	6
5	Frontend Overview	7
6	Installation	8
7	Project Structure	9
7.1	Deploy Panel Layout	9
8	Conclusion	11
8.1	Summary	11
8.2	Key Achievements	11
8.3	Future Scope	11

Introduction

1.1 Overview

Devi is a production-grade, AI-powered DevOps automation platform that transforms infrastructure management through natural language interaction. Built with a futuristic, cinematic interface inspired by Iron Man's Jarvis, Devi enables users to deploy, monitor, and manage Docker containers and Kubernetes clusters using voice commands or text input. The platform integrates real-time system monitoring, anomaly detection, CI/CD pipeline management, and intelligent container orchestration into a unified dashboard experience.

1.2 Motivation

Modern DevOps operations require expertise in multiple tools — Docker, Kubernetes, CI/CD pipelines, monitoring systems, and cloud platforms. This complexity creates a steep learning curve for new engineers and slows down experienced operators performing routine tasks. Devi addresses this challenge by providing a natural language interface that abstracts away command-line intricacies, allowing operators to manage infrastructure using plain English or voice commands.

1.3 Objectives

- Build an AI-powered platform that interprets natural language commands for infrastructure management
- Provide real-time system monitoring with anomaly detection and proactive alerting
- Enable voice-controlled container and cluster operations through a conversational assistant
- Implement safety mechanisms to prevent dangerous or unauthorized operations
- Create a cinematic, futuristic user interface that makes infrastructure management engaging and accessible

1.4 Scope

The platform covers Docker container lifecycle management (create, start, stop, restart, remove), Kubernetes cluster monitoring and pod management, natural language command parsing for automated deployments, real-time system metrics visualization, autonomous anomaly detection with proactive suggestions, CI/CD pipeline status tracking, and a conversational voice assistant with speech synthesis. The system is designed for both individual developers and enterprise DevOps teams.

System Architecture

2.1 Three-Tier Architecture

The platform follows a modern three-tier architecture separating concerns across presentation, application, and infrastructure layers:

- **Presentation Tier (Frontend):** Next.js 16 with React 19, Tailwind CSS v4, and Framer Motion animations — delivers a responsive, cinematic user interface across 14 routes including dashboards, chat, deployment panels, and system monitoring.
- **Application Tier (Backend):** Express.js with TypeScript, Socket.io for real-time communication, and Swagger/OpenAPI documentation — handles business logic, AI processing, NLP command parsing, and REST API routing.
- **Infrastructure Tier (Docker/K8s):** Docker daemon integration via Dockerode and Kubernetes API interaction via @kubernetes/client-node — executes actual infrastructure operations with real container deployment and cluster management.

The frontend communicates with the backend through HTTP REST endpoints for CRUD operations and WebSocket connections for real-time streaming of deployment logs, container updates, and system metrics. The backend services layer orchestrates all business logic including intent classification, command parsing, safety validation, and anomaly detection before executing infrastructure operations.

2.2 Service Ports

Port	Service	Description
3000	Frontend	Next.js dashboard with AI chat and deployment UI
4000	Backend API	Express.js REST API + Socket.io WebSocket
4000/api/docs	API Docs	Swagger UI for interactive exploration

2.3 Technology Stack

Category	Technology	Version
Frontend	Next.js / React / Tailwind CSS	16.2 / 19 / 4
Animations	Framer Motion	11
Backend	Express.js / TypeScript / Node.js	4 / 5 / 26
Docker SDK	Dockerode	4
Real-Time	Socket.io	4
Kubernetes	@kubernetes/client-node	0.21
Voice	Web Speech API	Browser native

Key Features

3.1 Natural Language Management

Devi's AI engine interprets commands into infrastructure actions:

"Deploy MongoDB on port 27017"	"Create Redis container"
"Restart nginx"	"Stop postgres"
"Show all containers"	"Deploy MERN stack"
"Check system health"	"Show CPU usage"
"Show pods in production"	"Scale web-frontend to 5"

3.2 AI Command Parser

The NLP parser (`services/parser.ts`) resolves intent through pattern matching:

- **Image Resolution:** `mongodb` → `mongo:7`, `nginx` → `nginx:alpine`
- **Port Extraction:** `"on port 27017"` → `27017:27017`
- **Stack Detection:** `"MERN stack"` triggers multi-service compose generation
- **Project Detection:** `package.json` → `Node.js`, `Cargo.toml` → `Rust`

3.3 Safe Execution Layer

All operations pass through `services/safety.ts`:

Check	Rule	Description
Image Whitelist	Allow-list	Only well-known images permitted
Privileged Mode	Blocked	<code>-privileged</code> containers rejected
Host Path Mounts	Blocked	<code>/etc</code> , <code>/var/run</code> , <code>/proc</code> prevented
Timeout	120s	Operations auto-terminated after 2 minutes
Name Validation	1-64 chars	Alphanumeric names only

3.4 Anomaly Detection

Type	Threshold	Detection
CPU Spike	>85% + 130% avg	Sudden utilization increase
Memory Leak	>90% + upward trend	Growth over 5+ samples
Crash-Loop	≥3 restarts/5min	Container restart cycling
High Latency	>500ms	Slow response times
Error Rate	>5% requests	Application error spike
Disk Full	>90%	Storage exhaustion warning

3.5 Voice Assistant

- Always-listening microphone on chat page — no wake word
- Silence detection processes audio at ~ 1.5 s pause
- Speech synthesis with calm cinematic tone
- Auto-restart after speaking for continuous conversation

3.6 Real-Time Deployment Streaming

Deployments stream live progress via WebSocket events. The client subscribes to deployment channels and receives real-time updates as containers are pulled, created, and started. This provides full visibility into the deployment lifecycle without polling.

```
Connection: ws://localhost:4000

deployment:start      "MERN stack deployment initiated"
deployment:progress   "Pulling mongo:7..."
deployment:progress   "Creating container mongodb..."
deployment:success    "MERN stack deployed successfully"
deployment:error      "Port 3000 is already in use"
docker:update         Container list refreshed
activity:new          Activity feed updated
```

3.7 Deployment Workflow

When a user issues a command like "Deploy MongoDB on port 27017", the following pipeline executes:

1. **Voice/Text Input:** Captured via Web Speech API or text input on the chat/deploy page, sent to `POST /api/docker/parse`
2. **NLP Parsing:** The parser extracts image name (`mongo:7`), port mapping (`27017:27017`), and container name (`mongodb`) from natural language
3. **Safety Validation:** The safety layer checks the image against the whitelist, validates the port range, and confirms the container name format
4. **Image Pulling:** Dockerode pulls the `mongo:7` image from Docker Hub with progress streamed via WebSocket
5. **Container Creation:** A new container is created with the specified port bindings and default environment variables
6. **Container Start:** The container is started and health-checked; success/failure is reported back through the UI
7. **Activity Logging:** The deployment event is recorded in the activity feed and the container list is refreshed

3.8 Frontend Components

Key reusable components include:

- **HolographicCard:** Glass-morphism card with animated gradient borders and backdrop blur
- **Sidebar:** Navigation sidebar with active state indicators and icon-based links
- **useVoiceAssistant:** Custom React hook integrating Web Speech API with silence detection (1500ms timeout) and speech synthesis for Jarvis-style responses
- **Deploy Panel:** Quick-deploy buttons for MongoDB, Redis, Nginx, PostgreSQL, Node.js, MySQL, plus MERN/LAMP/Redis stack templates with progress indicators

API Reference

4.1 Docker Endpoints

Method	Endpoint	Description
GET	/api/docker/containers	List all containers
POST	/api/docker/containers/deploy	Deploy a container
POST	/api/docker/containers/:id/{start,stop,restart}	Container lifecycle
GET	/api/docker/containers/:id/logs	Fetch container logs
POST	/api/docker/parse	NLP parse a deploy command
POST	/api/docker/create	Create container (pull + create)
POST	/api/docker/deploy	Full deploy (pull + create + start)
POST	/api/docker/compose	Deploy compose stacks
POST	/api/docker/containerize	Auto-detect and containerize
GET	/api/docker/logs/:id	Stream deployment logs

4.2 Kubernetes Endpoints

Method	Endpoint	Description
GET	/api/kubernetes/pods	List pods
GET	/api/kubernetes/nodes	List nodes
GET	/api/kubernetes/deployments	List deployments
GET	/api/kubernetes/namespaces	List namespaces
GET	/api/kubernetes/health	Cluster health check
POST	/api/kubernetes/deployments/:name/{restart,scale}	Deployment actions
DELETE	/api/kubernetes/pod/:ns/:name	Delete a pod
GET	/api/kubernetes/pod/:ns/:name/logs	Stream pod logs

4.3 AI and System Endpoints

Method	Endpoint	Description
POST	/api/ai/chat	Process natural language commands
GET	/api/ai/history	Chat history
GET	/api/system/stats	CPU, RAM, disk, network metrics
GET	/api/health	Server health check
POST	/api/terminal/execute	Execute shell command
POST	/api/terminal/validate	Validate command safety
POST	/api/agent/anomalies	Detect anomalies
POST	/api/agent/suggestions	Generate suggestions
POST	/api/agent/remember	Store agent memory

Frontend Overview

Route	Description
/	Landing page with Devi branding, video background
/login	Authentication (admin / admin123)
/dashboard	System metrics with CPU/RAM/disk charts
/chat	AI chat with voice input and typewriter responses
/deploy	Deployment panel with quick-deploy templates
/docker	Container create/start/stop/restart
/kubernetes	K8s cluster overview
/cicd	CI/CD pipeline management
/agent	Anomaly detection and suggestions
/terminal	Interactive web terminal
/logs	Centralized log viewer
/monitoring	Monitoring dashboards
/settings	Platform configuration

Installation

Prerequisites

Node.js v18+, Docker Desktop (optional), npm/yarn, modern browser.

Quick Start

```
git clone https://github.com/devi-platform/devi.git
cd devi

cd backend && npm install && cp .env.example .env
npm run build && npm start &

cd ../devi && npm install && cp .env.example .env.local
npm run build && npm start &

open http://localhost:3000    # Login: admin / admin123
```

Project Structure

devi/	
src/	# Frontend (Next.js)
app/	# 14 pages
page.tsx	Landing page
login/	Authentication
dashboard/	System metrics
chat/	AI chat + voice
deploy/	Deployment panel
docker/	Container mgmt
kubernetes/	K8s management
cicd/	CI/CD pipelines
agent/	Anomaly agent
terminal/	Web terminal
logs/	Log viewer
monitoring/	Monitoring
settings/	Configuration
components/	HolographicCard, Sidebar, Terminal
hooks/	useVoiceAssistant
lib/	api.ts, socket.ts, auth.ts
backend/	
src/	
index.ts	Server entry
routes/	auth, docker, deploy, kubernetes, ai, system, agent, cicd, terminal, logs
services/	docker, kubernetes, ai, parser, safety, system, agent, memory, auth, shell, logs
socket/	WebSocket event handlers
devi-hero/	Marketing landing page
Dockerfile	/ Dockerfile.frontend
.env.example	/ README.md
report.tex	This document

7.1 Deploy Panel Layout

The deploy page (/deploy) provides a comprehensive container deployment interface organized into three sections:

- **AI Input Bar:** A text input at the top where users type or speak deployment commands like "Deploy Redis" or "Create MongoDB container". The input is processed through the NLP parser and the result is displayed before execution.
- **Quick-Deploy Buttons:** Six one-click deployment buttons for popular service images — MongoDB, Redis, Nginx, PostgreSQL, Node.js, and MySQL. Each button immediately initiates pull, create, and start operations with sensible defaults.
- **Stack Deployments:** Three stack deployment buttons (MERN, LAMP, Redis Stack) that generate multi-service Docker Compose configurations and deploy all containers simultaneously with proper network configuration.

- **Container Management:** Below the deployment controls, a list of running containers displays status, ports, and uptime, with start/stop/restart action buttons and a logs viewer for each container.

Conclusion

8.1 Summary

Devi represents a paradigm shift in DevOps infrastructure management. By combining natural language processing, real-time streaming, and cinematic UI design, it makes complex infrastructure operations accessible through simple voice commands. The platform is production-ready, fully containerized, and designed for extensibility. Real-world testing has verified successful container deployment (mongo:7, Redis stacks) against a live Docker daemon, and the entire codebase compiles with zero TypeScript errors.

8.2 Key Achievements

- **Real Docker Integration:** Successfully deployed containers against a real Docker daemon via voice commands, with automatic image pulling, port mapping, and health verification
- **Zero TypeScript Errors:** The entire codebase compiles cleanly with no type errors across both frontend and backend
- **Natural Language Understanding:** NLP parser accurately resolves colloquial service names ("mongodb" → mongo:7), extracts port mappings from conversational phrases, and detects multi-service stack templates
- **Comprehensive Safety:** Multi-layer security system enforcing image whitelists, privileged container blocks, host path protection, and 120-second operation timeouts
- **Full Dashboard Ecosystem:** 14 frontend pages covering every aspect of infrastructure management with real-time metrics and cinematic UI

8.3 Future Scope

- **Helm Chart Management:** Kubernetes package management through natural language commands
- **Cloud Provider Integration:** AWS, Azure, and GCP resource management via Devi's AI interface
- **Autonomous Operations:** Self-healing infrastructure that detects and resolves issues without human intervention
- **Multi-Cluster Management:** Centralized management of multiple Kubernetes clusters across cloud providers
- **Advanced Analytics:** ML-based predictive analytics for capacity planning, cost optimization, and failure prediction